Material			
Course Code:			
Course Title:	Operating System		
Program:	BCA/Bscit/IIP	**Semester:**	III

Unit 2

Getting Started with Unix Commands

1. **What is Unix?**
2. **Architecture of UNIX Operating System**
3. **Features of Unix Operating System**
4. **Types of Shell in UNIX Operating System**

What is Unix?

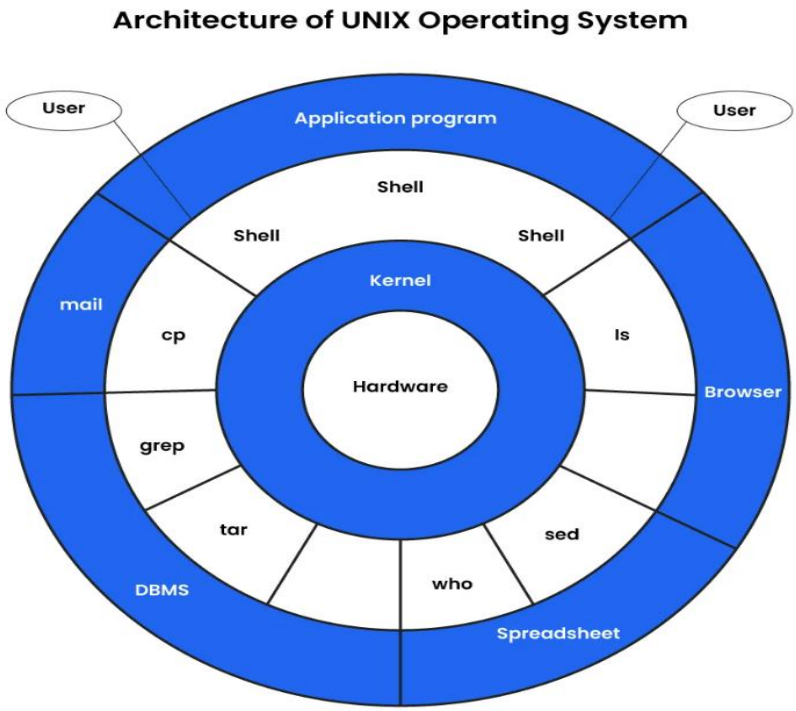


Dennis Ritchie (standing) and Ken Thompson working on a PDP-11 minicomputer

- UNIX full form is **UNiplexed Information and Computing System**, originally pronounced U-NICS.
- Unix is an operating system that can handle multiple users and processes at the same time.

- **Dennis Ritchie** and **Ken Thompson** of AT&T's Bell Labs developed Unix in the late 1960s and early 1970s.

Architecture of UNIX Operating System

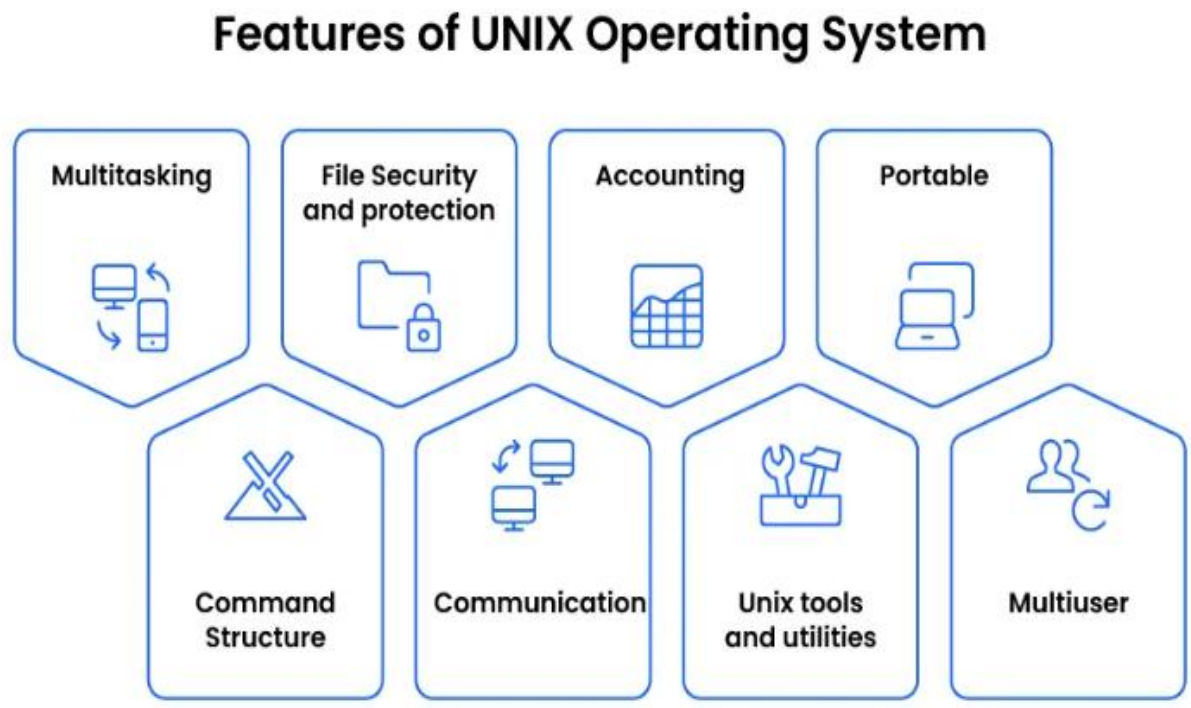


The Architecture of the UNIX system is divided into 4 major components

1. **Hardware Layer**
   - It is a repository of UNIX OS environment-related data useful for the processing of hardware.
2. **Kernel Layer**
   - The ' heart ' of the operating system.
   - It interfaces between Shell and Hardware.
   - It performs Low Level Task.
   - Eg: Device Management, Memory Management etc.
3. **Shell Layer**
   - The Shell is a collection of UNIX Commands.
   - The Shell acts as an interface between the user and the kernel.
   - The Shell is a Command Line Interpreter(CLI)–Translates the commands provided by the user and converts it into a language that is understood by the Kernel.
   - Only one Kernel running on the system, but several shells in action-one for each user who is logged in.
   - Eg: C Shell, Bourne Shell, Korn Shell etc

4. **Application Programs Layer**
   - The last layer of the Unix architecture is the Application Program layer.
   - As the name suggests, this outermost layer of Unix Architecture is responsible for executing the application programs.

Features of Unix Operating System



1. **Multitasking**
   - UNIX has the ability to launch many jobs or processes on the same terminal, with one running in the background and the other in the foreground.
2. **File Security and Protection**
   - On a multi-user system like UNIX, the biggest disadvantage is that UNIX is more open to security holes.
   - Hence, security and protection of files is of prime importance in UNIX.
   - UNIX security practices involve issuing usernames and passwords, authentication, file access permissions, and file encryption.
3. **Accounting**
   - UNIX uses system accounting tools to monitor each user's activities and resource usage.
   - These tools aid in keeping an eye on system performance, disc space, and CPU utilisation. Administrators have the ability to enforce disc quotas, monitor disc usage by user, and set space limits for each user.For this, commands like quota, df (disk free), and du (disk usage) are frequently used.
   - In order to ensure that users use system resources effectively and fairly, a superuser (like root) can control these resources and enforce regulations.

4. **Portability**
   - As much of the UNIX operating system is written in the C language and only a fraction is in the form of machine-dependent code, this guarantees portability of code to other computers. This is why UNIX is extremely portable.
5. **Command Structure**
   - Commands in UNIX are easy to learn and use.
   - Basic UNIX commands are case-sensitive (i.e. UNIX recognizes capital, small and capitalized letters as different letters) and typically typed in lowercase.
   - Example: cp, mv etc.
6. **Communication**
   - The UNIX based operating systems provides the communication through methods like write command, mail command, etc.
7. **Supports Multi-User Environment**
   - Multiple users can share system resources like CPU, memory, and storage. thanks to the UNIX operating system's support for multiple-users.
   - Every user can function independently and log in from a separate terminal, either locally or remotely.
   - To distribute system resources effectively, UNIX employs time-sharing mechanisms, allocating CPU time to each user's processes according to a scheduling algorithm.
   - This makes UNIX OS perfect for settings that need multi-user access, such as servers, university labs, and enterprise systems, by guaranteeing that several jobs can run simultaneously without affecting one another.
8. **Unix tools and Utilities**
   - There are different kind of tools in UNIX OS.
   - These are: grep, sed, awk command in unix, etc.
   - These values are useful for compiler, network, or remote services.

Types of Shell in UNIX Operating System

1. **Bourne Shell (sh)/** Bourne-Again Shell (bash)**: The original Unix shell**
   - ✓ Released: 1979 (Unix V7)
   - ✓ Developer: Stephen Bourne, Bell Labs
   - ✓ It is the original UNIX shell.
   - ✓ It is faster and more preferred.
   - ✓ It lacks features for interactive use like the ability to recall previous commands.
   - ✓ It also lacks built-in arithmetic and logical expression handling.
   - ✓ It is the default shell for Solaris OS.
2. **Bash Shell (bash): An improved version of the Bourne shell**
   - ✓ Released: 1989
   - ✓ Developer: Brian Fox, GNU Project
   - ✓ It is compatible with the Bourne shell.
   - ✓ It includes features from Korn and Bourne shell.
3. **C Shell (csh): Similar to C programming syntax**
   - ✓ Released: 1978 (BSD Unix)
   - ✓ Developer: Bill Joy, UC Berkeley
   - ✓ It incorporated features such as aliases and command history.

- ✓ It includes helpful programming features like built-in arithmetic and C-like expression

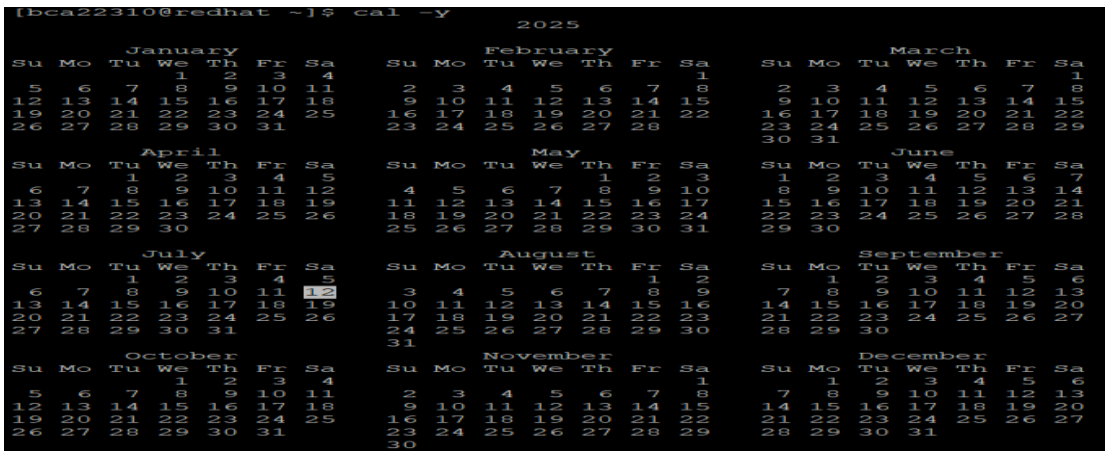
4. **Korn Shell (ksh): Advanced scripting with better performance**
   - ✓ Released: 1983; integrated into System V Release 4 (circa 1988)
   - ✓ Developer: David Korn, Bell Labs
   - ✓ So it supports everything in the Bourne shell.
   - ✓ It has interactive features.
   - ✓ It includes features like built-in arithmetic and C-like arrays, functions, and string-manipulation facilities.
   - ✓ It is faster than C shell.
   - ✓ It is compatible with script written for C shell.

Basic Command

1. cal

Description:	The cal command displays a calendar of the specified year or month. By default, it shows the calendar for the current month. You can specify a particular month or year to display.
Syntax:	cal [options] [month] [year]
Example:	

2. cal -y

Description:	This command will display the current year along with all months.
Syntax:	Cal -y
Example:	

3. cal year

Description:	This command will display any specific year along with all months of that year.
Syntax:	Cal [year]
Example:	

```
[bca22310@redhat ~]$ cal 2020
                                2020

      January               February                March
Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa
          1  2  3  4                     1      1  2  3  4  5  6  7
 5  6  7  8  9 10 11   2  3  4  5  6  7  8      8  9 10 11 12 13 14
12 13 14 15 16 17 18   9 10 11 12 13 14 15     15 16 17 18 19 20 21
19 20 21 22 23 24 25  16 17 18 19 20 21 22     22 23 24 25 26 27 28
26 27 28 29 30 31     23 24 25 26 27 28 29     29 30 31
                      30

       April                  May                   June
Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa
          1  2  3  4                  1  2         1  2  3  4  5  6
 5  6  7  8  9 10 11   3  4  5  6  7  8  9      7  8  9 10 11 12 13
12 13 14 15 16 17 18  10 11 12 13 14 15 16     14 15 16 17 18 19 20
19 20 21 22 23 24 25  17 18 19 20 21 22 23     21 22 23 24 25 26 27
26 27 28 29 30        24 25 26 27 28 29 30     28 29 30
                      31

        July                 August               September
Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa
          1  2  3  4                     1            1  2  3  4  5
 5  6  7  8  9 10 11   2  3  4  5  6  7  8      6  7  8  9 10 11 12
12 13 14 15 16 17 18   9 10 11 12 13 14 15     13 14 15 16 17 18 19
19 20 21 22 23 24 25  16 17 18 19 20 21 22     20 21 22 23 24 25 26
26 27 28 29 30 31     23 24 25 26 27 28 29     27 28 29 30
                      30 31

       October              November              December
Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa
             1  2  3   1  2  3  4  5  6  7            1  2  3  4  5
 4  5  6  7  8  9 10   8  9 10 11 12 13 14      6  7  8  9 10 11 12
11 12 13 14 15 16 17  15 16 17 18 19 20 21     13 14 15 16 17 18 19
18 19 20 21 22 23 24  22 23 24 25 26 27 28     20 21 22 23 24 25 26
25 26 27 28 29 30 31  29 30                    27 28 29 30 31
```

4. cal month year

Description:	Display calendar of specific month and year.
Syntax:	cal [month][year]
Example:	

```
[bca22310@redhat ~]$ cal 03 2020
     March 2020
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31
```

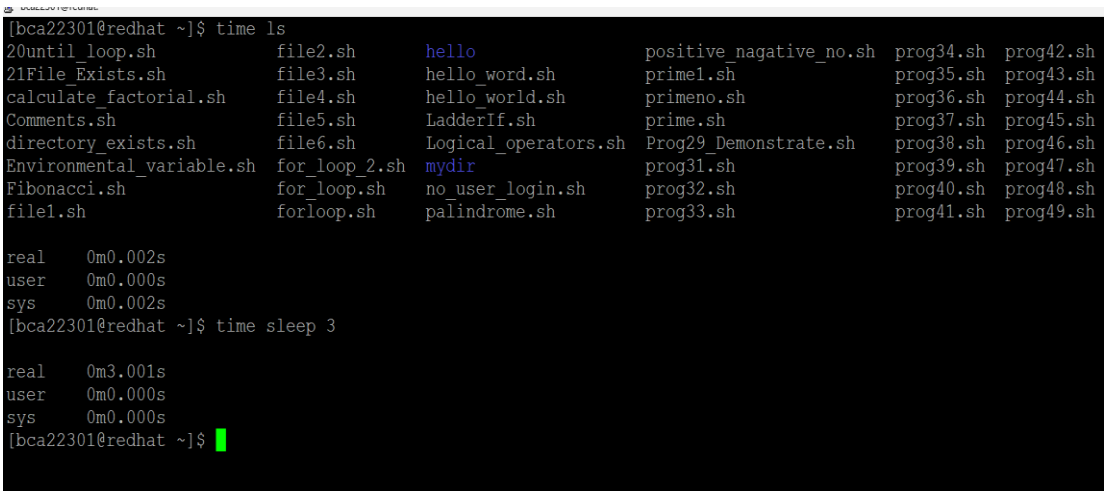
5. Date

Description:	This Command will display the current date.
Syntax:	date
Example:	Command: [bca22310@redhat ~]$ date Mon Aug 4 16:58:52 IST 2025 [bca22310@redhat ~]$ date +%a Mon [bca22310@redhat ~]$ date +%A Monday [bca22310@redhat ~]$ date +%b Aug [bca22310@redhat ~]$ date +%B August [bca22310@redhat ~]$ date +%c Mon 04 Aug 2025 04:59:17 PM IST [bca22310@redhat ~]$ date +%D 08/04/25 [bca22310@redhat ~]$ date +%j 216 [bca22310@redhat ~]$ date +%p PM [bca22310@redhat ~]$ date +%T 17:00:03 [bca22310@redhat ~]$ date +%t [bca22310@redhat ~]$ date +%V 32
Options	%a Abbreviated weekday(Tue). %A Full weekday(Tuesday). %b Abbreviated month name(Jan). %B Full month name(January). %c Country-specific date and time format..

	% D Date in the format %m/%d/%y. %j Julian day off year (001-366). %p String to indicate a.m. or p.m. %T Time in the format %H:%M:%S. %t Tab space. %V Week number in year (01-52), start week on Monday

6. Time

Description:	The time command is used to measure **how long a command takes to execute** (i.e., its execution time). **Real time** ? actual elapsed time, including time spent waiting for I/O and other processes. **User time** ? amount of CPU time spent in user-mode code. **System time** ? amount of CPU time spent in kernel-mode code.
Syntax:	time [options] command [arguments]
Example:	**This will list the files in the current directory and also display how much time it took to execute the ls command.** command : time ls **This command makes the system sleep for 3 seconds, and time shows how long it took.** command : time sleep 3



7. ipconfig

Description:	The ipconfig command is used to configure network interfaces. It can display information about all network interfaces or a specific one.
Syntax:	ipconfig [interface]
Example:	ipconfig /all

```
C:\Users\Seminar-A>ipconfig/all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : Bhavya
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Realtek PCIe GbE Family Controller
   Physical Address. . . . . . . . . : 64-4E-D7-05-FB-07
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

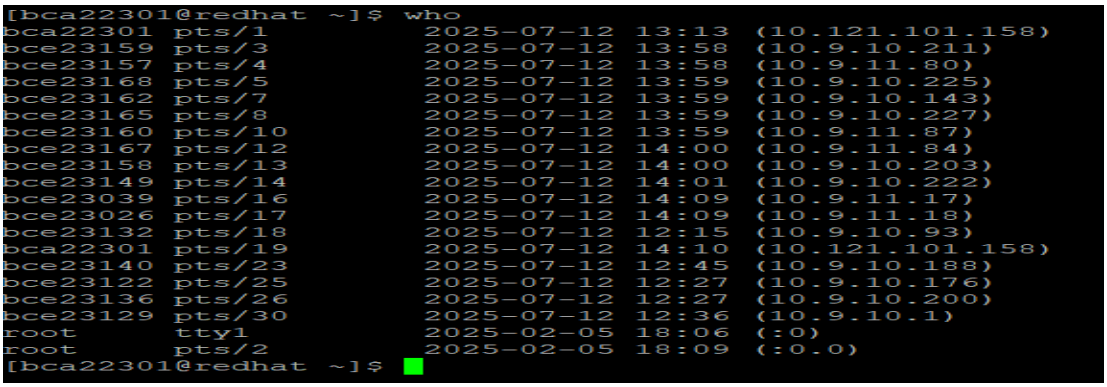
Wireless LAN adapter Local Area Connection* 3:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #3
   Physical Address. . . . . . . . . : CE-47-40-95-19-00
```

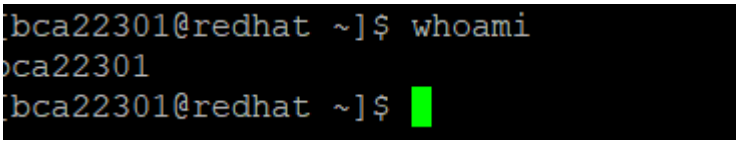
Login Related Command

8. who

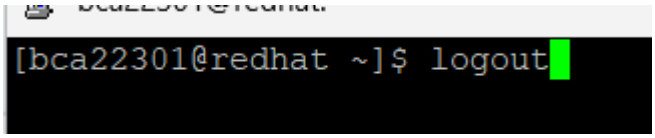
Description:	Displays information about users currently logged in.
Syntax:	who

Example:	

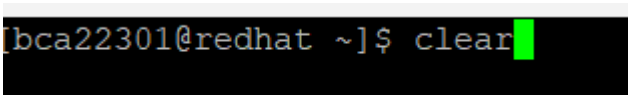
9. whoami

Description:	Shows the current username.
Syntax:	whoami
Example:	

10.logout

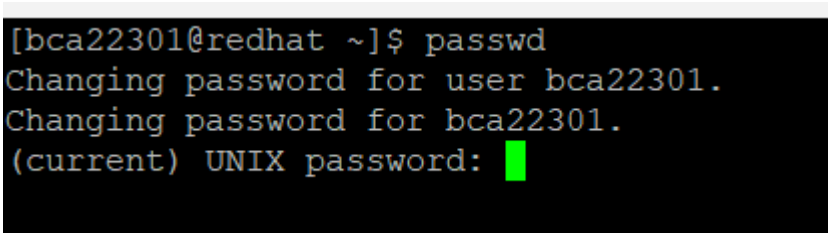
Description:	Log out from the current terminal session.
Syntax:	logout
Example:	

11. clear

Description:	This command simply clears the terminal screen
Syntax:	clear
Example:	

12. password(passwd)

Description:	Changes the user's password.

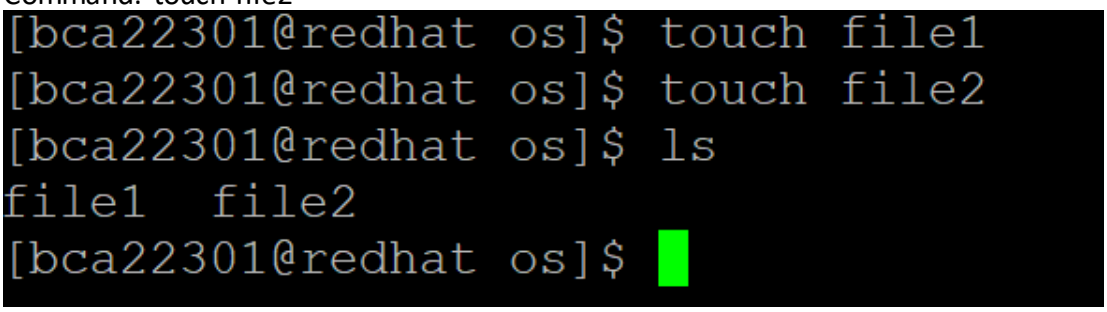
Syntax:	passwd
Example:	

File and Directory related command

1. **Creating Files (touch , cat)**
2. **Including in file (cp ,mv)**
3. **Listing Files and Directories (ls)**
4. **Permissions (chmod)**
5. **removing a file (rm)**
6. **Directory Related Command(pwd , mkdir, rmdir, cd)**
7. **File related command (wc)**
8. **Viewing Files (head, tail)**

1. Creating Files (touch , cat)

1. touch [single]

Description:	This command is used to create new files one at a time.
Syntax:	touch filename
Example:	Command: touch file1 Command: touch file2 

2. touch multiple(create multiple file)

Description:	This command is used to create multiple new files at a same time.

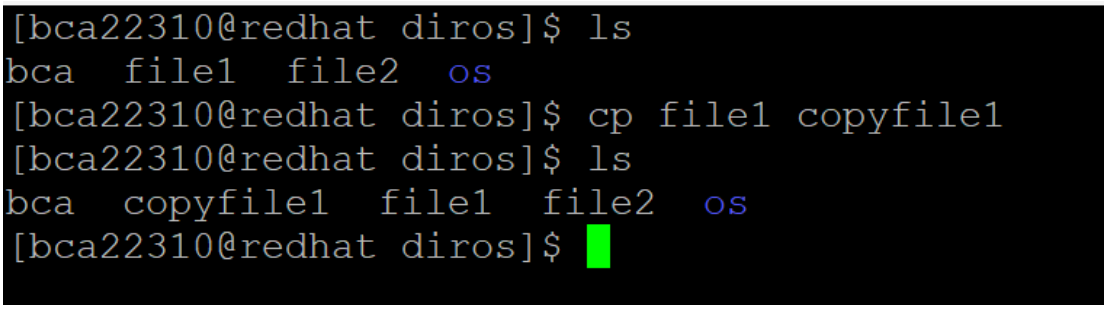
Syntax:	touch filename1 filename2 filename3
Example:	Command: touch file3 file4 file5 [bca22301@redhat os]$ touch file3 file4 file5 [bca22301@redhat os]$ ls file1 file2 file3 file4 file5 [bca22301@redhat os]$

2.cat

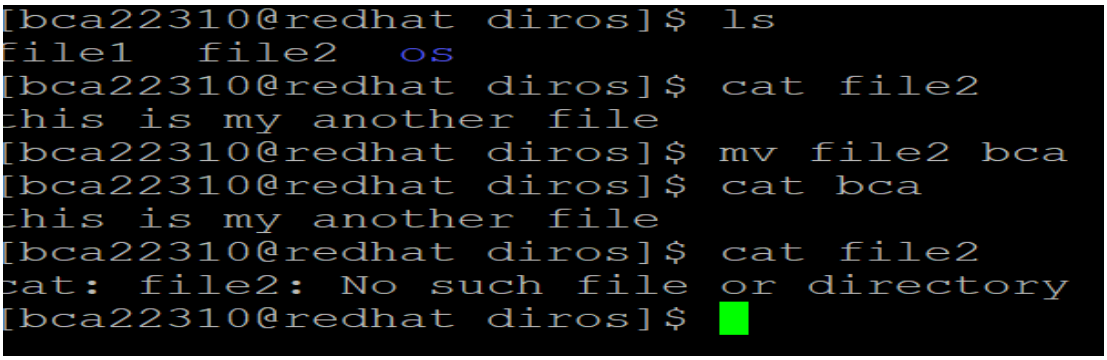
Description:	Used for adding content to a particular file. Uses with other options: 1. Display the Contents of a File [cat filename] 2. Concatenate Multiple Files [cat file1 file2] 3. Create a New File and Add Content [cat > filename new] 4. Append Content to an Existing File [cat >> existing file] 5. Redirect Contents of a File to Another File [cat file1 > file2] 6. Append Contents of a File to Another File [cat file1 >> file2]
Syntax:	cat [options] file
Example:	command : cat > file1 (Create a New File and Add Content and Display) [bca22310@redhat diros]$ cat > file1 Hello this is my first file Now print only one file [bca22310@redhat diros]$ cat file1 Hello this is my first file Now print only one file command: cat > file2(file 2 display contain) cat file1 file2(combine and display file contain) [bca22310@redhat diros]$ cat > file2 this is my another file [bca22310@redhat diros]$ cat file1 file2 Hello this is my first file Now print only one file this is my another file [bca22310@redhat diros]$

2. Including in file (cp , rm ,mv)

1. cp(copy command)

Description:	The cp command copies files and directories from one location to another. It can be used to copy a single file, multiple files, or entire directories.
Syntax:	cp [options] source destination
Example:	

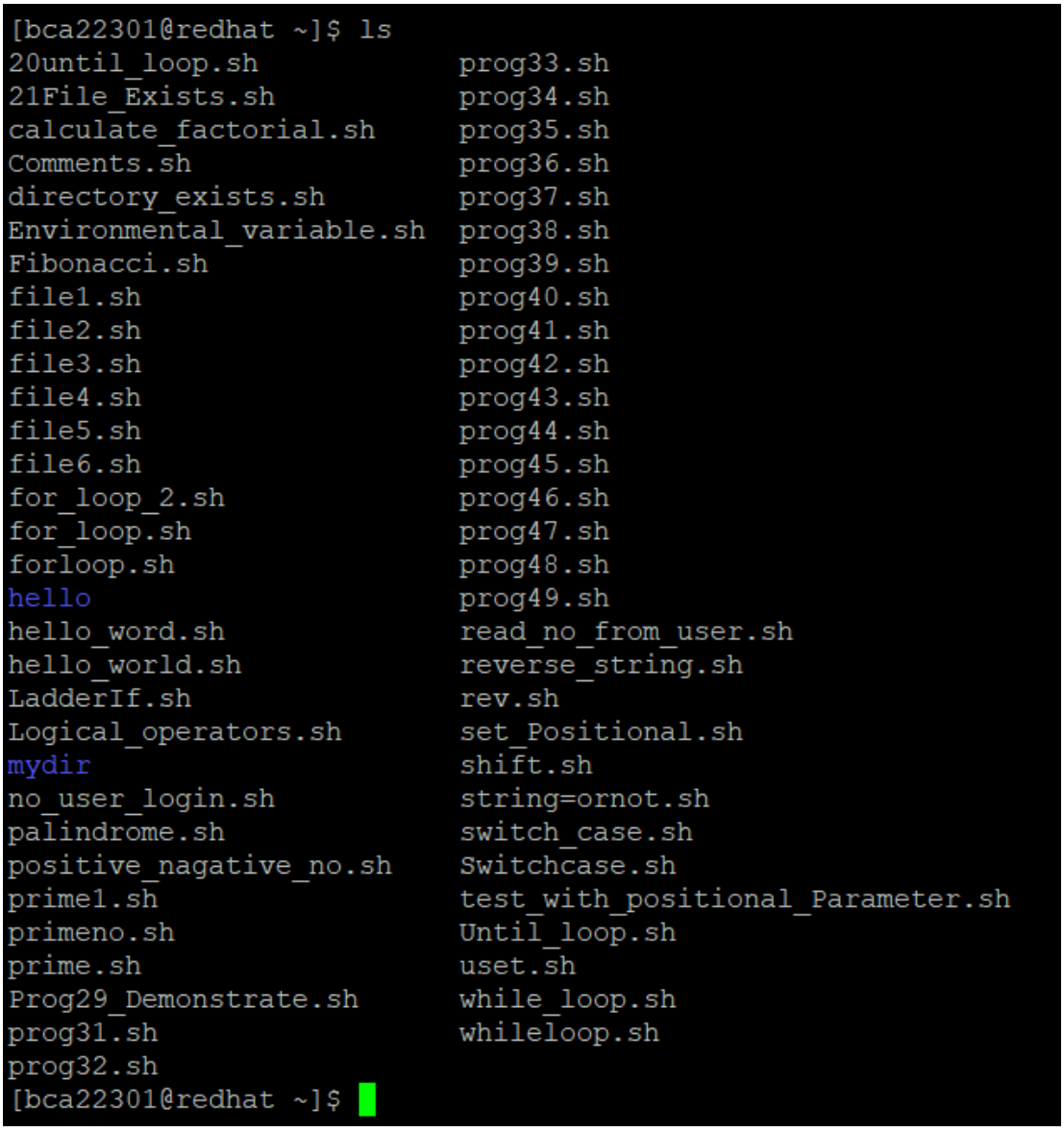
3. mv

Description:	This Command is used to move or rename a file.
Syntax:	mv [options] source destination
Example:	command mv file2 file3 

3. Listing Files and Directories (ls)

1. ls

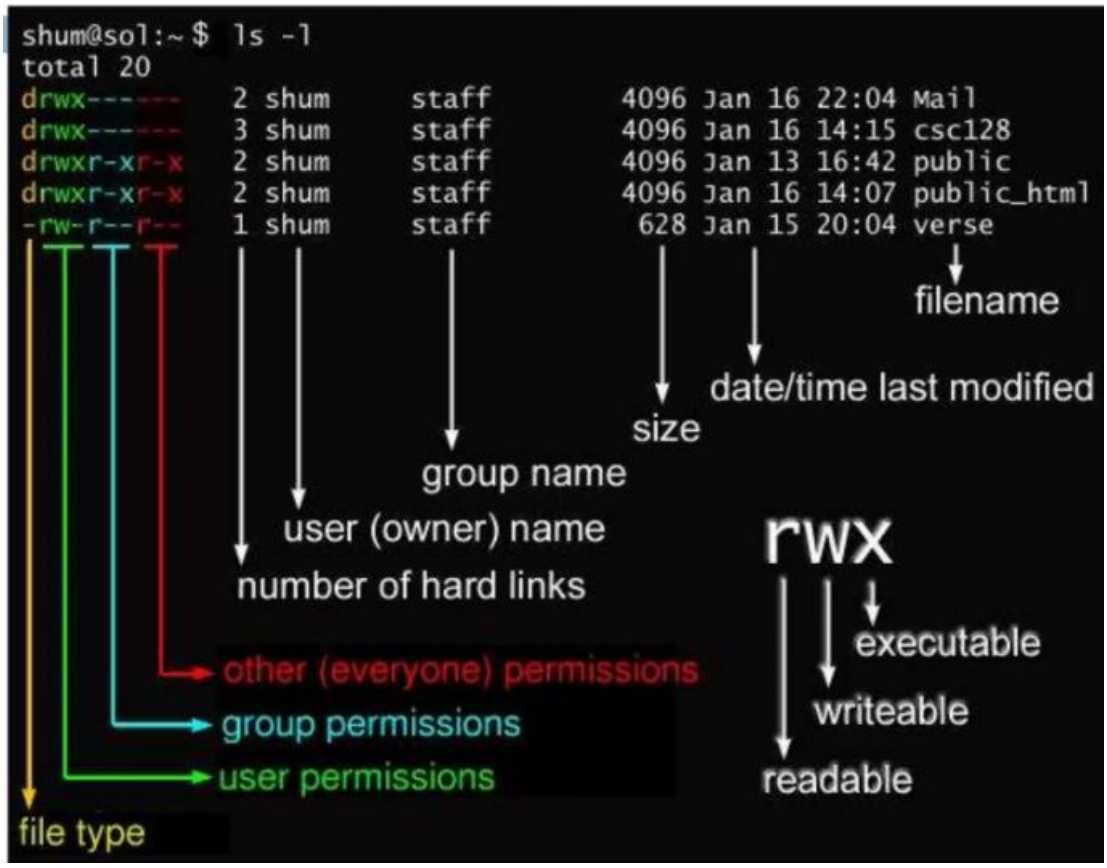
Description:	The ls command simply lists the directory contents.
Syntax:	ls [options] [directory] • ls Lists files and directories in the current directory. • ls -l A more complete list with permissions

	• ls -a Hidden files were included
Example:	

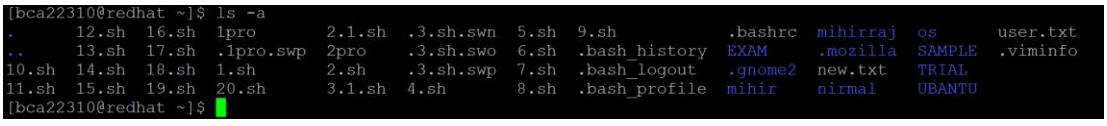
```
[bca22301@redhat ~]$ ls
20until_loop.sh             prog33.sh
21File_Exists.sh            prog34.sh
calculate_factorial.sh      prog35.sh
Comments.sh                 prog36.sh
directory_exists.sh         prog37.sh
Environmental_variable.sh   prog38.sh
Fibonacci.sh                prog39.sh
file1.sh                    prog40.sh
file2.sh                    prog41.sh
file3.sh                    prog42.sh
file4.sh                    prog43.sh
file5.sh                    prog44.sh
file6.sh                    prog45.sh
for_loop_2.sh               prog46.sh
for_loop.sh                 prog47.sh
forloop.sh                  prog48.sh
hello                       prog49.sh
hello_word.sh               read_no_from_user.sh
hello_world.sh              reverse_string.sh
LadderIf.sh                 rev.sh
Logical_operators.sh        set_Positional.sh
mydir                       shift.sh
no_user_login.sh            string=ornot.sh
palindrome.sh               switch_case.sh
positive_nagative_no.sh     Switchcase.sh
prime1.sh                   test_with_positional_Parameter.sh
primeno.sh                  Until_loop.sh
prime.sh                    uset.sh
Prog29_Demonstrate.sh       while_loop.sh
prog31.sh                   whileloop.sh
prog32.sh
[bca22301@redhat ~]$
```

2. ls -l

Description:	The -l command option give detailed information about the directory contents in a columnar format that includes size, modified date and time, file or directory name and owner of the file, and its permission. 1st column – File/directory permissions. 2nd column – Number of links. 3rd column – Name of the owner. 4th column – Name of the group that the file belongs to. 5th column – File size in bytes. 6th column to 8th column – Last modification date. 9th column – File / Directory name.

Syntax:	ls -l
Example:	

3. ls –a

Description:	-a option will display both hidden files and directories.
Syntax:	ls -a . refers to the current directory. .. refers to the parent directory.
Example:	

4. changing Permissions (chmod)

1. chmod

Description:	The chmod command in Unix/Linux is used to **change the file permissions** (i.e., who can read, write, or execute a file). **chmod Types:**

1.Symbolic Mode (Text Format)

user type	Operator:	Permission:
u = user (owner) g = group o = others a = all (user + group + others)	+ → add permission - → remove permission = → set exact permission	r = read w = write x = execute

example

```
chmod u+x file.txt   # Add execute to user

chmod g-w file.txt   # Remove write from group
chmod o=r file.txt   # Set only read permission for others
chmod a+rw file.txt  # Add read/write for all
```

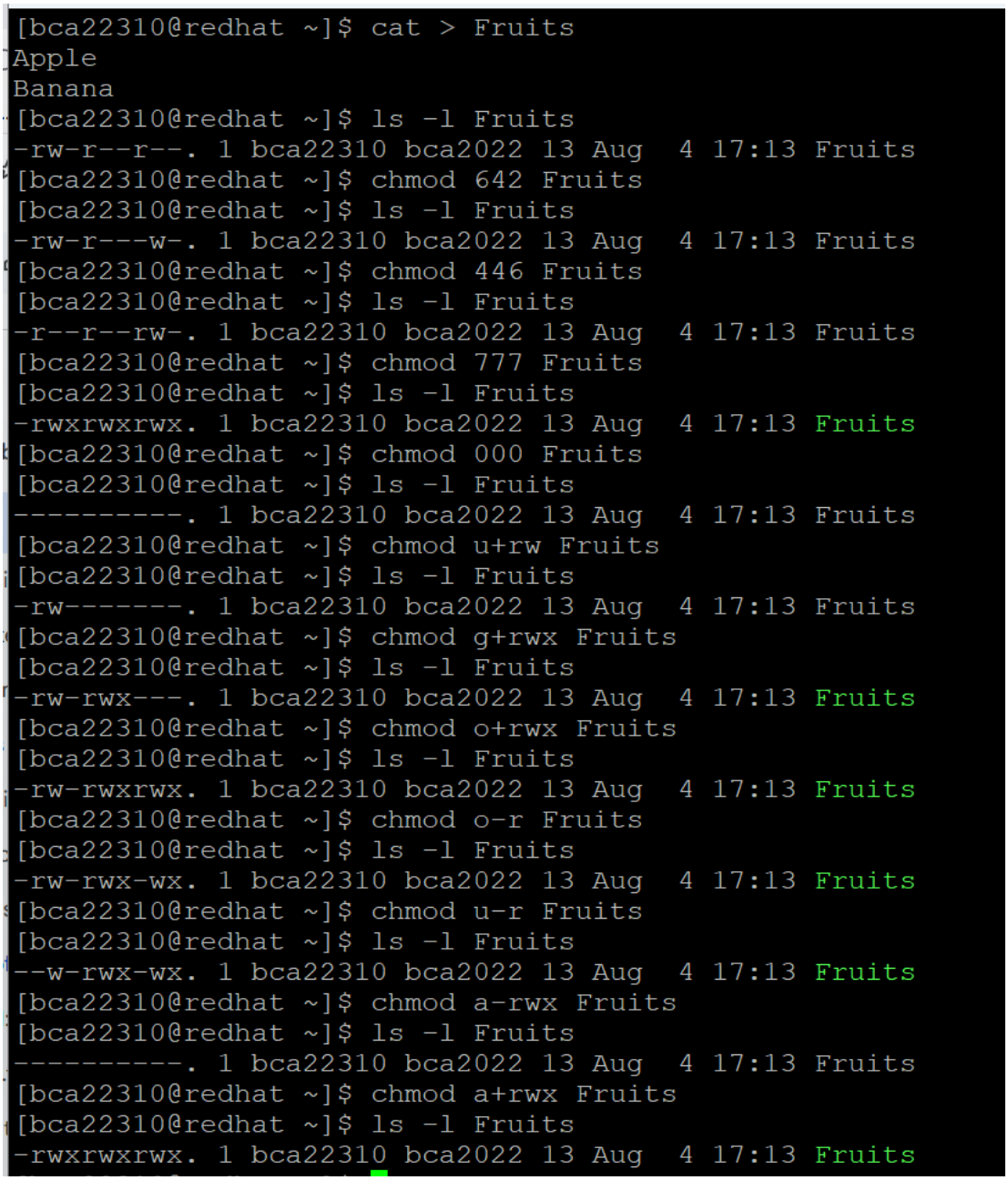
2.Numeric (Octal) Mode

Permission	Value
r = read	4
w = write	2
x = execute	1

example

```
chmod 755 file.txt
```

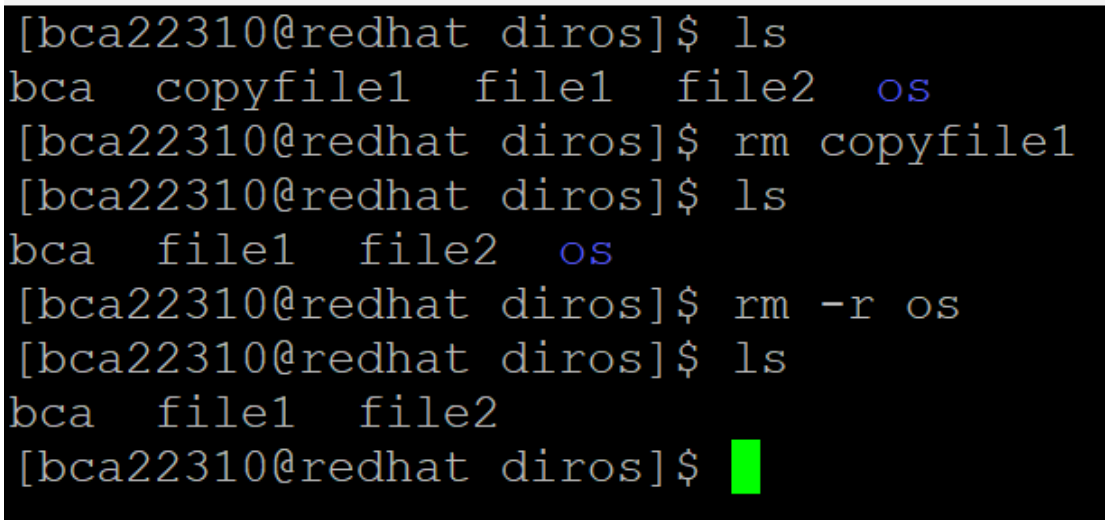
Syntax:	chmod [options] mode file

Example:	

removing a file (rm)

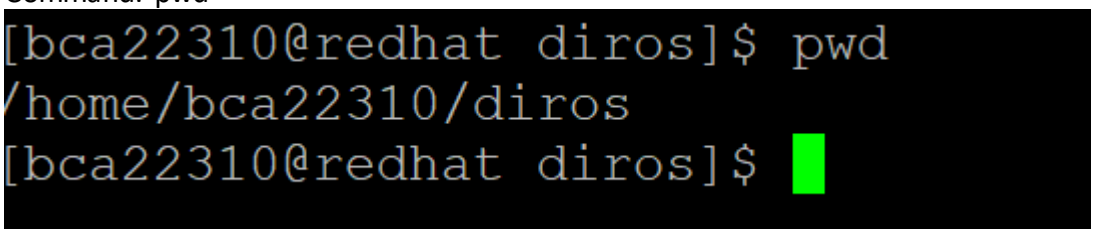
4. rm(remove file)

Description:	This command is used to remove files.
Syntax:	rm [options] file
Example:	Commands: rm copyfile1 removefile rm -r d1 remove the directory



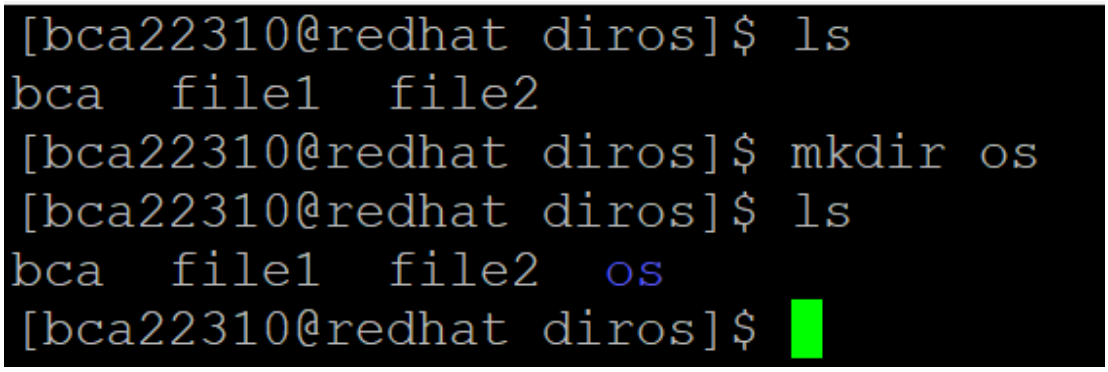
Directory Related Command(pwd , mkdir, rmdir, cd)

1. pwd(display current working directory (path))

Description:	Displaying your current directory name (Print working directory).
Syntax:	pwd
Example:	Command: pwd 

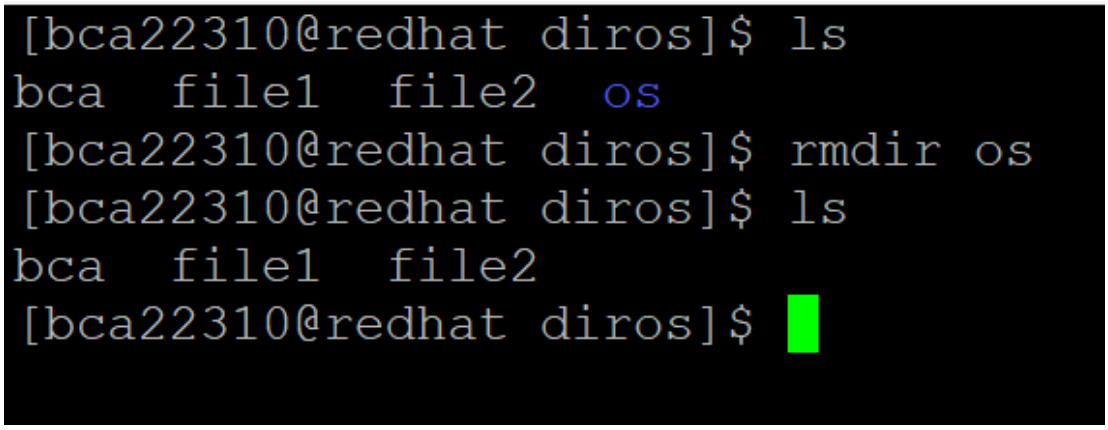
2. mkdir

Description:	This command is used to create a new directory. We can also create number of subdirectories with one mkdir command.
Syntax:	mkdir [options] directory
Example:	Command: mkdir os



3. **rmdir**

Description:	It is used to delete/remove a directory and its subdirectories.
Syntax:	rmdir [options..] Directory
Example:	Command: rmdir dir1



4. cd

Description:	cd (change directory) is used to change from one current working directory to another directory.
Syntax:	cd [directory_name] cd <dirname> cd or cd ~ – to home directory. cd / – to root directory. cd - – Switches to the previous directory. cd .. – Moves up one directory level.
Example:	Command: cd OS

```
[bca22310@redhat diros]$ mkdir os
[bca22310@redhat diros]$ ls
file1  file2  os
[bca22310@redhat diros]$ cd os
[bca22310@redhat os]$ cd ~
[bca22310@redhat ~]$ cd /
[bca22310@redhat /]$ cd -
/home/bca22310
[bca22310@redhat ~]$ cd diros
[bca22310@redhat diros]$ cd os
[bca22310@redhat os]$ cd ..
[bca22310@redhat diros]$
```

File related command (wc)

1. wc(word count)

Description:	This command will display the number of lines, number of words, number of bytes in a particular file.
Syntax:	wc
Example:	Command: wc file2 Lines (15): The first number (15) represents the number of lines in the file. Words (15): The second number (15) represents the number of words in the file. Bytes (90): The third number (90) represents the number of bytes in the file.

Viewing Files (head, tail)

1. head

Description:	The head command displays the first few lines of a file or files. By default, it shows the first 10 lines.
Syntax:	head [options] [file...]

Example:	Command:head -5 file1 Cat > file1 (Entered 20 lines in file1) Head file1 (This command by default display first 10 lines of File)

```
ten[bca22310@redhat diros]$ head -5 file3
one
two
three
four
five
[bca22310@redhat diros]$
```

2. tail

Description:	The tail command displays the last few lines of a file. By default, it shows the last 10 lines
Syntax:	tail filename
Example:	

```
[bca22310@redhat diros]$ ls
bca  file  file1  file2  file3
[bca22310@redhat diros]$ tail file3
one
two
three
four
five
six
seven
eight
nine
ten[bca22310@redhat diros]$ tail -3 file3
eight
nine
ten[bca22310@redhat diros]$
```

Why Redirection commands ?

Redirection commands are fundamental tools in Unix-like operating systems that provide extensive control over the input and output of commands.Redirection commands enable users to streamline their workflow by automating the handling of input and output.Redirection facilitates efficient data management.

Redirection Commands	Description

">"	Used to redirect output to a file, creating a new file if it doesn't exist or overwriting the existing file if it does.
	Example: Command: cat > newfile.txt
	`[bca22310@redhat ~]$ ls`
	`10.sh 14.sh 18.sh 20.sh 3.1.sh 7.sh EXAM nirmal UBANTU`
	`11.sh 15.sh 19.sh 2.1.sh 4.sh 8.sh mihir os user.txt`
	`12.sh 16.sh 1pro 2pro 5.sh 9.sh mihirraj SAMPLE`
	`13.sh 17.sh 1.sh 2.sh 6.sh diros new.txt TRIAL`
	`[bca22310@redhat ~]$ cd os`
	`[bca22310@redhat os]$ ls`
	`file1 file2`
	`[bca22310@redhat os]$ cat >newfile.txt`
	`hello user`
	`this command will let you enter content to your file`
	`thank you hace a nice day`
	`[bca22310@redhat os]$`

">>"	Used to redirect output to a file, but appending the output to the end of the file instead of overwriting it.
	Example: Command : cat >> newfile.txt
	`[bca22310@redhat os]$ ls`
	`file1 file2 newfile.txt`
	`[bca22310@redhat os]$ cat >> newfile.txt`
	`this is my new file .`
	`[bca22310@redhat os]$ cat newfile.txt`
	`hello user`
	`this command will let you enter content to your file`
	`thank you hace a nice day`
	`this is my new file .`
	`[bca22310@redhat os]$`

"<"	This is standard input redirection, cat command will take the input from “file.txt” and print it to the terminal screen. Example : cat < newfile.txt

```
[bca22310@redhat os]$ cat < newfile.txt
hello user
this command will let you enter content to your file
thank you hace a nice day
this is my new file .
[bca22310@redhat os]$
```

1. Pipe Operator

Description:	The **pipe (|)** command is used to **pass the output of one command as input to another command**. It allows chaining multiple commands to perform complex operations efficiently.
Syntax:	command1 | command2

Example:	**Count number of lines in a file**
	command :cat file.txt | wc -l
	`[bca22310@redhat os]$ cat file1| wc -l`
	`2`
	command: cat file.txt

2. grep

Description:	**grep is a search tool. Grep, short for "global regular expression print", is a command used for searching and matching text patterns in files contained in the regular expressions.**
Syntax:	grep [options] pattern [file...]

Example:	1. **Search a word in a file**
	`localhost:~/os# ls`
	`book.lst`
	`localhost:~/os# cat book.lst`
	`101 | database management | ssn | 10/09/2001 | ajit gupta`
	`102 | system analysis | r & r corp | 12/03/1990 | ajit gupta`
	`103 | unix | r & r crop | 12/04/2009 | jain`
	`104 | c and c++ | ssn | 05/04/2000| jain`
	`localhost:~/os# grep 'ssn' book.lst`
	`101 | database management | ssn | 10/09/2001 | ajit gupta`
	`104 | c and c++ | ssn | 05/04/2000| jain`
	`localhost:~/os# grep -c 'ssn' book.lst`
	`2`
	`localhost:~/os#`

Working with column and row

1. cut

Description:	Used to filter particular column or fields out of a file.

Syntax:	cut [options] [filename]
	
	`-f (--fields=LIST)`
	`-c (--characters=LIST)`
	`-d (--delimiter)`

Example:	`localhost:~/os# cat book.lst`
	`101 | database management | ssn | 10/09/2001 | ajit gupta`
	`102 | system analysis | r & r corp | 12/03/1990 | ajit gupta`
	`103 | unix | r & r crop | 12/04/2009 | jain`
	`104 | c and c++ | ssn | 05/04/2000| jain`
	`localhost:~/os# cut -c 7-27 book.lst`
	`database management |`
	`system analysis | r &`
	`unix | r & r crop | 1`
	`c and c++ | ssn | 05/`
	`localhost:~/os# cut -d | -f 2,5 book.lst`
	` database management | ajit gupta`
	` system analysis | ajit gupta`
	` unix | jain`
	` c and c++ | jain`
	`localhost:~/os#`

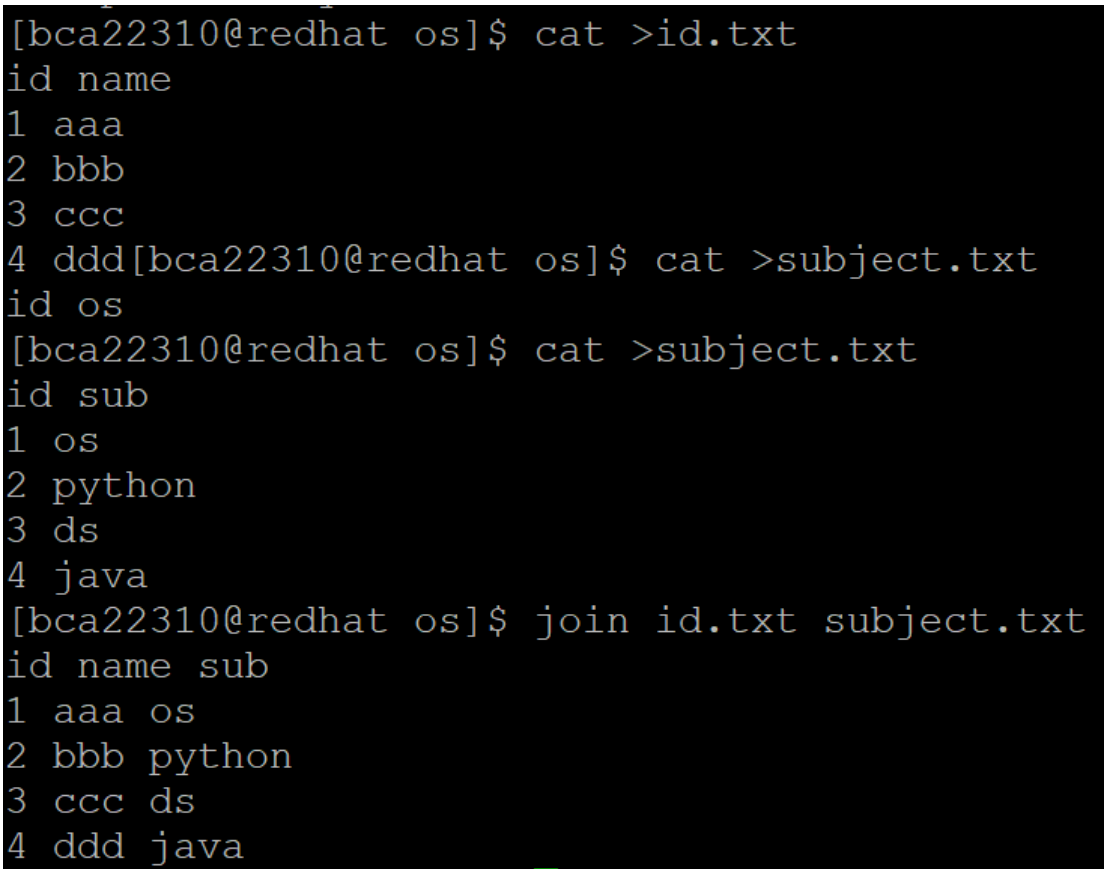
2. paste

Description:	The **paste command** is used to **merge lines from multiple files side-by-side (vertically)**. It's useful when you have separate files for different columns and want to combine them into a single file or view.
Syntax:	paste [options] file1 file2 ...
Example:	**Step 1: Create 3 Separate Files** **1. id.txt** *2. name.txt 3. subject.txt* command paste id.txt name.txt subject.txt

```
localhost:~/os# cat > id.txt
1
2
3
4
5
localhost:~/os# cat > subject.txt
os
java
linux
ctop
python
localhost:~/os# cat > name.txt
aaa
bbb
ccc
ddd
eee
localhost:~/os# paste id.txt subject.txt name.txt
1       os      aaa
2       java    bbb
3       linux   ccc
4       ctop    ddd
```

3. join

Description:	The join command is used to **combine two files based on a common field (like a key)** — similar to a **SQL JOIN**. Note:Unlike paste (which joins side-by-side by line number), join joins based on **matching values in sorted files**. **Important Rules:** 1. **Both files must be sorted** on the join field. 2. **Default join field is the first column**. 3. Use -1 and -2 to specify custom fields to join on.
Syntax:	join file1 file2

Example:	

Tools for sorting

26-sort

Description:	The sort command is used to **arrange lines** of text files in a **specific order**, such as alphabetical or numerical.
Syntax:	`sort [OPTION]... [FILE]...`
Example:	1. **Sort Alphabetically** **2. Sort in Reverse Order**

```
[bca22310@redhat os]$ cat sortlist
apple
banana
mango
banana
orange
[bca22310@redhat os]$ sort sortlist
apple
banana
banana
mango
orange
[bca22310@redhat os]$ sort -r sortlist
orange
mango
banana
banana
apple
[bca22310@redhat os]$ sort -u sortlist
apple
banana
mango
orange
[bca22310@redhat os]$
```

27- uniq

Description:	The uniq command in Linux is used to **remove duplicate lines** from a **sorted** text file or input.
Syntax:	uniq [options] filename
Example:	1. **Remove Consecutive Duplicates** **-uniq file.txt** **Notice: Only consecutive duplicates are removed. The last apple remains because it's not next to the others.** **2. With Sort (Best Practice)**

sort file.txt | uniq

Now all duplicates are removed correctly.

3. Show Only Repeated Lines

sort file.txt | uniq -d

4. Show Only Unique Lines (Not Repeated)

sort file.txt | uniq -u

5. Count Each Line

sort file.txt | uniq -c

```
[bca22310@redhat os]$ cat sortlist
apple
banana
mango
banana
orange
[bca22310@redhat os]$ uniq sortlist
apple
banana
mango
banana
orange
[bca22310@redhat os]$ sort sortlist|uniq
apple
banana
mango
orange
[bca22310@redhat os]$ sort sortlist|uniq -d
banana
[bca22310@redhat os]$ sort sortlist|uniq -u
apple
mango
orange
[bca22310@redhat os]$ sort sortlist|uniq -c
      1 apple
      2 banana
      1 mango
      1 orange
```

COMPARING FILES : cmp, diff

1. cmp — Compare Byte by Byte

Description:	cmp compares **two files byte-by-byte** and tells the **first difference**.

Syntax:	cmp file1.txt file2.txt

Example:	**Example 1: Same files**
	`[bca22310@redhat os]$ cat > file1.txt`
	`hello`
	`word`
	`[bca22310@redhat os]$ cp file1.txt file2.txt`
	`[bca22310@redhat os]$ cat file2.txt`
	`hello`
	`word`
	`[bca22310@redhat os]$ cmp file1.txt file2.txt`
	`[bca22310@redhat os]$`
	(no output) → files are identical
	Example 2: Different files
	`[bca22310@redhat os]$ cat file2.txt`
	`hello`
	`unix`
	`[bca22310@redhat os]$ cmp file1.txt file2.txt`
	`file1.txt file2.txt differ: byte 7, line 2`
	`[bca22310@redhat os]$`

2. diff — Compare Line by Line

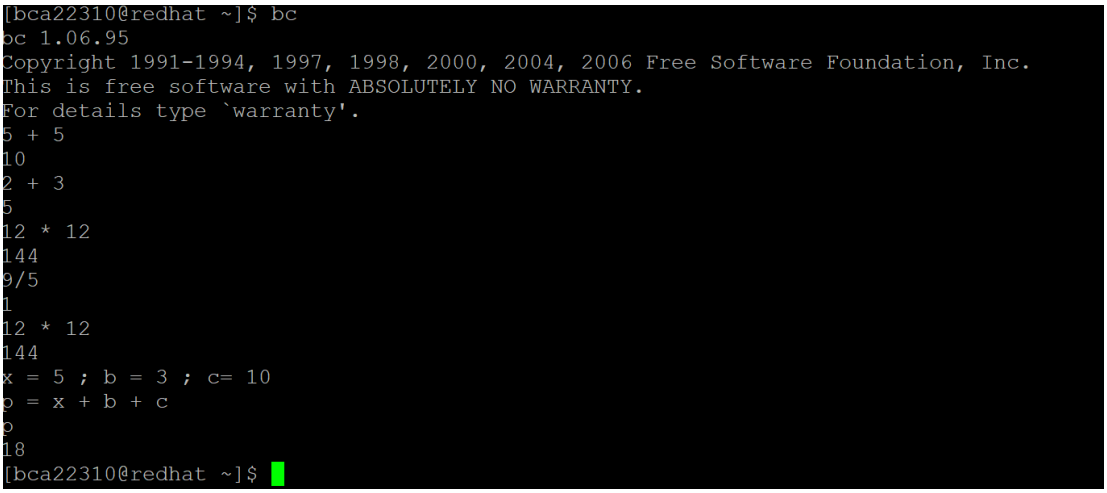
Description:	diff compares **two files line-by-line** and shows how to **make them the same**.
Syntax:	diff file1.txt file2.txt

Example:	`[bca22310@redhat os]$ diff file1.txt file2.txt`
	`2c2`
	`< word`
	`---`
	`> unix`
	`[bca22310@redhat os]$`

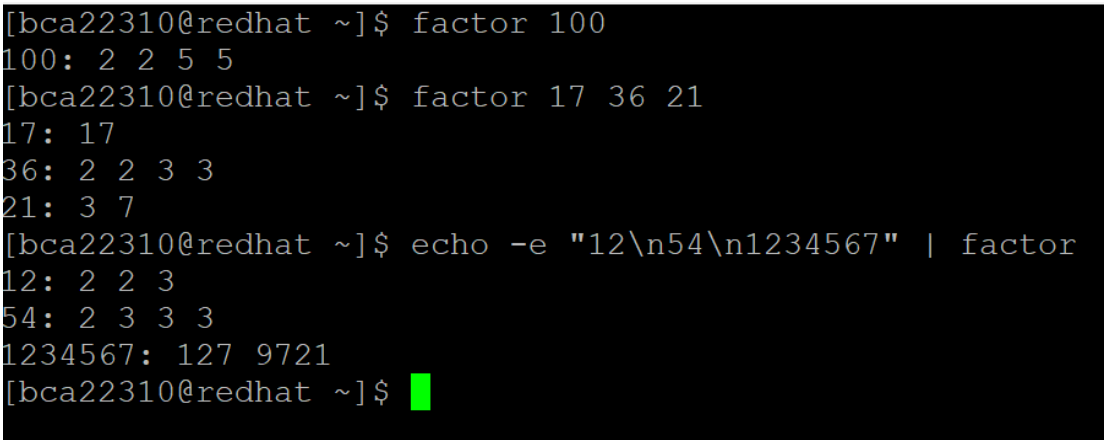
Mathematical command

1. bc

Description:	The bc command is used for precision calculation. You are more likely to use it for floating point operations in bash scripts.

Syntax:	bc (basic calculator)
Example:	

2. factor

Description:	The factor command **breaks down an integer into its prime factors**. If you don't give it a number on the command line, it **reads numbers from standard input (i.e., your keyboard or piped data)**
Syntax:	factor [OPTION] [NUMBER]
Example:	

21-for basic calculator(+,-,*,/)

Description:	This command lets you use your terminal window as a calculator. Expr stands for "expression" .
Syntax:	Command Syntax: expr number operator number
Example:	Command:

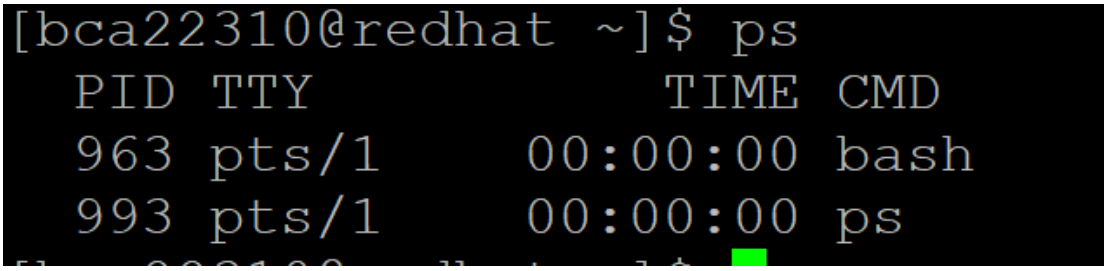
	expr 4 + 4 expr 4 - 4 expr 4 * 4 expr 4 / 4	or	echo "5 + 3" | bc # Addition → 8 echo "10 - 4" | bc # Subtraction → 6 echo "6 * 7" | bc # Multiplication → 42 echo "20 / 4" | bc # Division → 5 echo "20 % 3" | bc # Modulus → 2

22-logical operation(>,<,>=,<=,=,)

Description:	Logical expressions containing&&, !, <, <=, >, >=, =, != can be evaluated. The command returns "1" if the expression evaluates to true and "0" if it evaluates to false. All the logical operators are escaped with a backward slash (\).
Syntax:	Command Syntax: expr number operator number
Example:	Command: expr 4 \> 5 expr 4 \>= 5 expr 4 \< 5 expr 6 \<= 5 expr 4 \= 4 expr 4 \!= 5

Process related command :ps, kill, wait, sleep

1. ps -Show Process Status

Description:	Displays information about the currently running processes.
Syntax:	ps
Example:	

2. kill -Stop a Running Process

Description:	Sends a signal to a process, usually to stop (terminate) it.
Syntax:	kill PID
Example:	1. First, find the process ID: ps 2. Then stop a process: kill 2625 3. To force kill : kill -9 2625

```
[bca22310@redhat os]$ ps
  PID TTY          TIME CMD
 2492 pts/1    00:00:00 bash
 2625 pts/1    00:00:00 ps
[bca22310@redhat os]$ kill -9 2625
-bash: kill: (2625) - No such process
[bca22310@redhat os]$ kill 2625
-bash: kill: (2625) - No such process
[bca22310@redhat os]$ ps
  PID TTY          TIME CMD
 2492 pts/1    00:00:00 bash
 2708 pts/1    00:00:00 ps
[bca22310@redhat os]$
```

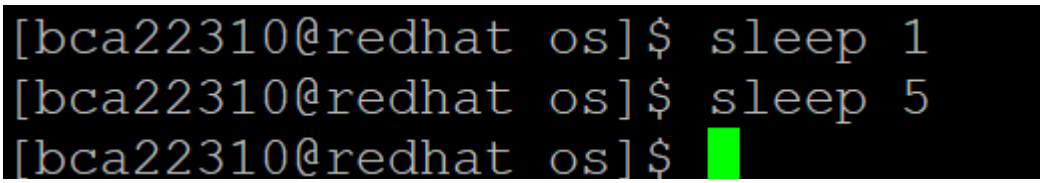
3. wait — *Wait for a Process to Finish*

	Waits until the specified background process finishes.
Syntax:	wait [PID]
Example:	

```
[bca22310@redhat os]$ sleep &
[1] 2758
[bca22310@redhat os]$ sleep: missing operand
Try `sleep --help' for more information.
wait
[bca22310@redhat os]$ echo done
done
```

4. sleep — *Pause Execution*

Description:	Pauses the script or command for a specific amount of time.
Syntax:	sleep [seconds]

Example:	

[bca22310@redhat os]$ sleep 1
[bca22310@redhat os]$ sleep 5
[bca22310@redhat os]$